GA GUARDIAN

Manifest

Protocol Review

Security Assessment

November 7th, 2025

Summary

Audit Firm Guardian

Prepared By Osman Ozdemir, Minato Namakazi, Michael Lett, Ali Shehab, Jordan Raphael

Client Firm Manifest Finance

Final Report Date November 7, 2025

Audit Summary

Manifest engaged Guardian to review the security of their Manifest Finance Protocol. From the 9th of September to the 17th of September, a team of 5 auditors reviewed the source code in scope. All findings have been recorded in the following report.

Confidence Ranking

Given the number of non-critical issues detected and code changes following the main review, Guardian assigns a Confidence Ranking of 3 to the protocol. Guardian advises the protocol to address issues thoroughly and consider a targeted follow-up audit depending on code changes. For detailed understanding of the Guardian Confidence Ranking, please see the rubric on the following page.

✅ Verify the authenticity of this report on Guardian's GitHub: https://github.com/guardianaudits

Guardian Confidence Ranking

Confidence Ranking	Definition and Recommendation	Risk Profile
5: Very High Confidence	Codebase is mature, clean, and secure. No High or Critical vulnerabilities were found. Follows modern best practices with high test coverage and thoughtful design. **Recommendation:** Code is highly secure at time of audit. Low risk of latent critical issues.	0 High/Critical findings and few Low/Medium severity findings.
4: High Confidence	Code is clean, well-structured, and adheres to best practices. Only Low or Medium-severity issues were discovered. Design patterns are sound, and test coverage is reasonable. Small changes, such as modifying rounding logic, may introduce new vulnerabilities and should be carefully reviewed. **Recommendation:** Suitable for deployment after remediations; consider periodic review with changes.	0 High/Critical findings. Varied Low/Medium severity findings.
3: Moderate Confidence	Medium-severity and occasional High-severity issues found. Code is functional, but there are concerning areas (e.g., weak modularity, risky patterns). No critical design flaws, though some patterns could lead to issues in edge cases. **Recommendation:** Address issues thoroughly and consider a targeted follow-up audit depending on code changes.	1 High finding and ≥ 3 Medium. Varied Low severity findings.
2: Low Confidence	Code shows frequent emergence of Critical/High vulnerabilities (~2/week). Audit revealed recurring anti-patterns, weak test coverage, or unclear logic. These characteristics suggest a high likelihood of latent issues. **Recommendation:** Post-audit development and a second audit cycle are strongly advised.	2-4 High/Critical findings per engagement week.
1: Very Low Confidence	Code has systemic issues. Multiple High/Critical findings (≥5/week), poor security posture, and design flaws that introduce compounding risks. Safety cannot be assured. **Recommendation:** Halt deployment and seek a comprehensive re-audit after substantial refactoring.	≥5 High/Critical findings and overall systemic flaws.

Table of Contents

Project Overview

Project Summary

Project Name	Manifest
Language	Solidity
Codebase	https://gitlab.com/manifest-finance/ush
Commit(s)	Main Review commit: 280c9bdd29c85c39352d87cb50f3af5594c98b56 Remediation Review commit: 81234f25bd856aefd55eae38a127360c28dcf15f

Audit Summary

Delivery Date	November 7, 2025
Audit Methodology	Static Analysis, Manual Review, Test Suite, Contract Fuzzing

Vulnerability Summary

Vulnerability Level	Total	Pending	Declined	Acknowledged	Partially Resolved	Resolved
Critical	0	0	0	0	0	0
High	1	0	0	0	0	1
Medium	6	0	0	2	0	4
Low	26	0	0	8	0	18
Info	28	0	0	6	0	22

Audit Scope & Methodology

```
contract,source,total,comment
ush/contracts/ush/USAToken.sol,27,48,15
ush/contracts/ush/USHToken.sol,106,207,70
ush/contracts/ush/USHTokenStorage.sol,13,24,8
ush/contracts/sush/AdminControl.sol,42,89,37
ush/contracts/sush/Silo.sol,18,39,16
ush/contracts/sush/StakedUSH.sol,65,144,54
ush/contracts/sush/StakedUSHBase.sol,139,297,112
ush/contracts/solmate-erc20-upgradeable/SolmateERC20StorageLib.sol,21,33,9
ush/contracts/solmate-erc20-upgradeable/SolmateERC20Upgradeable.sol,107,228,83
ush/contracts/hooks/AuthHook.sol,92,138,25
ush/contracts/chainalysis/MockChainalysisOracle.sol,26,63,29
ush/contracts/chainalysis/SanctionsList.sol,120,227,78
ush/contracts/chainalysis/SanctionsListStorage.sol,15,28,10
ush/contracts/authmanager/AuthManager.sol,265,468,130
ush/contracts/authmanager/AuthManagerStorage.sol,16,31,12
ush/contracts/oracle/USHPriceOracle.sol,145,273,82
ush/contracts/oracle/USHPriceOracleStorage.sol,20,38,15
source count: {
  total: 2375,
  source: 1237,
  comment: 785,
  single: 459,
  block: 326,
  mixed: 4,
  empty: 357,
  todo: 0,
  blockEmpty: 0,
  commentToSourceRatio: 0.6345998383185125
}
```

Audit Scope & Methodology

Vulnerability Classifications

Severity	Impact: *High*	Impact: *Medium*	Impact: *Low*
Likelihood: *High*	🔴 Critical	🟠 High	🟡 Medium
Likelihood: *Medium*	🟠 High	🟡 Medium	🟢 Low
Likelihood: *Low*	🟡 Medium	🟢 Low	🟢 Low

Impact

High Significant loss of assets in the protocol, significant harm to a group of users, or a core functionality of the protocol is disrupted.

Medium A small amount of funds can be lost or ancillary functionality of the protocol is affected. The user or protocol may experience reduced or delayed receipt of intended funds.

Low Can lead to any unexpected behavior with some of the protocol's functionalities that is notable but does not meet the criteria for a higher severity.

Likelihood

High The attack is possible with reasonable assumptions that mimic on-chain conditions, and the cost of the attack is relatively low compared to the amount gained or the disruption to the protocol.

Medium An attack vector that is only possible in uncommon cases or requires a large amount of capital to exercise relative to the amount gained or the disruption to the protocol.

Low Unlikely to ever occur in production.

Audit Scope & Methodology

Methodology

Guardian is the ultimate standard for Smart Contract security. An engagement with Guardian entails the following:

- Two competing teams of Guardian security researchers performing an independent review.
- A dedicated fuzzing engineer to construct a comprehensive stateful fuzzing suite for the project.
- An engagement lead security researcher coordinating the 2 teams, performing their own analysis, relaying findings to the client, and orchestrating the testing/verification efforts.

The auditing process pays special attention to the following considerations:

- Testing the smart contracts against both common and uncommon attack vectors.
- Assessing the codebase to ensure compliance with current best practices and industry standards.
- Ensuring contract logic meets the specifications and intentions of the client.
- Cross-referencing contract structure and implementation against similar smart contracts produced by industry leaders.
- Thorough line-by-line manual review of the entire codebase by industry experts. Comprehensive written tests as a part of a code coverage testing suite.
- Contract fuzzing for increased attack resilience.

Invariants Assessed

During Guardian's review of Manifest, fuzz-testing was performed on the protocol's main functionalities. Given the dynamic interactions and the potential for unforeseen edge cases in the protocol, fuzz-testing was imperative to verify the integrity of several system invariants.

Throughout the engagement the following invariants were assessed for a total of 10,000,000+ runs with a prepared fuzzing suite.

ID	Description	Tested	Passed	Remediation	Run Count
AM-01	KYC Status Should Be True After grantKYC	✅	✅	✅	10m+
AM-02	Ban Status Should Not Change After grantKYC	✅	✅	✅	10m+
AM-03	KYC Status Should Be False After revokeKYC	✅	✅	✅	10m+
AM-04	Ban Status Should Not Change After revokeKYC	✅	✅	✅	10m+
AM-05	KYC Status Should Be True For All Accounts After batchGrantKYC	✅	✅	✅	10m+
AM-06	Ban Status Should Not Change After batchGrantKYC	✅	✅	✅	10m+
AM-07	KYC Status Should Be False For All Accounts After batchRevokeKYC	✅	✅	✅	10m+
AM-08	Ban Status Should Not Change After batchRevokeKYC	✅	✅	✅	10m+
AM-09	Ban Status Should Be True After banUser	✅	✅	✅	10m+
AM-10	Ban Status Should Be False After removeBan	✅	✅	✅	10m+

Invariants Assessed

ID	Description	Tested	Passed	Remediation	Run Count
AM-11	Ban Status Should Be True For All Accounts After batchBan	✅	✅	✅	10m+
AM-12	Ban Status Should Be False For All Accounts After batchRemoveBan	✅	✅	✅	10m+
AM-13	Account Should Be In manifestAccounts after addManifestAccount	✅	✅	✅	10m+
AM-14	Account Should Not Be In manifestAccounts after removeManifestAccount	✅	✅	✅	10m+
INV-IUC-01	Incremental update count should increase by 1 after each successful price update	✅	✅	✅	10m+
INV-BP-01	Current price should always be within the defined min and max price bounds	✅	✅	✅	10m+
INV-UT-01	lastUpdatedTimestamp should be greater or equal to after a successful price update	✅	✅	✅	10m+
INV-PUDEMC-01	Price change percentage should not exceed maxChangePercentage when updated by priceUpdater	✅	✅	✅	10m+
INV-PUDEMUI-01	Price update should not occur before minUpdateInterval has passed from last update when updated by priceUpdater	✅	✅	✅	10m+
INV-PU	Ensures that each price update reflects an actual price change	✅	✅	✅	10m+
INV-PAC	Ensure that a Pending Manager proposal always has a corresponding request timestamp, and vice versa	✅	✅	✅	10m+

Invariants Assessed

ID	Description	Tested	Passed	Remediation	Run Count
INV-UAM	AuthManager can only update to previously pendingAuthManager	✅	✅	✅	10m+
STAKE-01	Deposit/mint should increase StakedUSH shares of the receiver	✅	✅	✅	10m+
STAKE-02	Deposit/mint should increase StakedUSH total supply	✅	✅	✅	10m+
STAKE-03	Deposit/mint should increase asset balance of the StakedUSH contract	✅	✅	✅	10m+
STAKE-04	Deposit/mint should decrease asset balance of the user	✅	✅	✅	10m+
STAKE-05	cooldownAssets/cooldownShares should decrease StakedUSH shares of the user	✅	✅	✅	10m+
STAKE-06	cooldownAssets/cooldownShares should decrease StakedUSH total supply	✅	✅	✅	10m+
STAKE-07	cooldownAssets/cooldownShares should decrease asset balance of the StakedUSH contract	✅	✅	✅	10m+
STAKE-08	cooldownAssets/cooldownShares should increase asset balance of the Silo contract	✅	✅	✅	10m+
STAKE-09	unstake should increase asset balance of the receive	✅	✅	✅	10m+
STAKE-10	unstake should decrease asset balance of the Silo contract	✅	✅	✅	10m+
STAKE-11	unstake should not change the total supply of StakedUSH	✅	✅	✅	10m+
STAKE-12	Withdraw/redeem should decrease StakedUSH shares of the user	✅	✅	✅	10m+

Invariants Assessed

ID	Description	Tested	Passed	Remediation	Run Count
STAKE-13	Withdraw/redeem should decrease StakedUSH total supply	✅	✅	✅	10m+
STAKE-14	Withdraw/redeem should decrease asset balance of the StakedUSH contract	✅	✅	✅	10m+
STAKE-15	Withdraw/redeem should increase asset balance of the receiver	✅	✅	✅	10m+
INV-CIO	Ensure cooldown duration is zero when a user withdraws from the vault	✅	✅	✅	10m+
INV-UAC	Ensures the unstaked amount reflected in the silo's total holdings matches the user's underlying amount recorded during cooldown	✅	✅	✅	10m+
INV-UCE	Ensures that the cooldown period has elapsed before allowing an unstake operation	✅	✅	✅	10m+
INV-URUS	Ensures that after an unstake operation, all user cooldown fields are reset to zero	✅	✅	✅	10m+
INV-CAC	Ensures that the staked amount reflected in the silo's total holdings matches the user's underlying amount recorded during cooldown	✅	✅	✅	10m+
INV-CSBMA	Checks that the burned share amount correspond to the assets moved into cooldown	✅	✅	✅	10m+
INV-RINU	Rescue shouldn't be called with USH Token	✅	✅	✅	10m+
INV-LDU	Ensures the lastDistributionTimestamp is incrementing	✅	✅	✅	10m+
INV-VAU	Ensures the vestingAmount got updated	✅	✅	✅	10m+

Invariants Assessed

ID	Description	Tested	Passed	Remediation	Run Count
INV-TIB	transferInRewards should increase the asset balance of StakedUSH contract	✅	✅	✅	10m+
INV-RFZB	Ensures the address from has no balance after the redistribution	✅	✅	✅	10m+
INV-RTB	Ensures the address to gets the redistributed balance	✅	✅	✅	10m+

Findings & Resolutions

ID	Title	Category	Severity	Status
H-01	AuthHook's Authentication Can Be Bypassed	Trust Assumptions	High	Pending
M-01	Bypass KYC Check By Transferring	Validation	Medium	Acknowledged
M-02	sUSH Cannot Be Redistributed	Validation	Medium	Pending
M-03	Price Decimal Mismatch In USHPriceOracle	Unexpected Behavior	Medium	Pending
M-04	Unauthorized Pool Initialization And Donations	Access Control	Medium	Pending
L-01	Invalid Permits Due To Version Mismatch	Validation	Low	Pending
L-02	Inconsistent Behavior At Unstake	Best Practices	Low	Pending
L-03	No Cancellation Mechanism	Informational	Low	Acknowledged
L-04	Unstake Cooldown Is Temporarily Denied	DoS	Low	Acknowledged
L-05	Discrepancy Between Similar Functions	Unexpected Behavior	Low	Pending
L-06	Unauthorized Users Can Perform Transfer	Access Control	Low	Pending
L-07	initializeVault Can Be Frontrunned	Frontrunning	Low	Pending
L-08	requiresMultiSigApproval Misses Interval Check	Validation	Low	Pending

Findings & Resolutions

ID	Title	Category	Severity	Status
L-09	Lost ECDSA Checks Enable Signature Malleability	Validation	Low	Pending
L-10	Reward Dilution Possible During transferInReward	MEV	Low	Acknowledged
L-11	Missing Validation In Auth Batch Functions	Validation	Low	Pending
L-12	Circulating Supply Misreported	Unexpected Behavior	Low	Pending
L-13	Checks Can Be Bypassed	Warning	Low	Acknowledged
L-14	setAuthManager Restarts Timelock	Unexpected Behavior	Low	Pending
L-15	hasValidAuth Does Not Account For Sanctions	Validation	Low	Pending
L-16	MIN_PRICE And MAX_PRICE Limit Price Updates	Best Practices	Low	Pending
L-17	New Cooldown Resets cooldownEnd	Best Practices	Low	Acknowledged
I-01	priceUpdater Can Set expectedIndex = Uint256.max	Best Practices	Info	Pending
I-02	Move _exceedsMaxChange Inside priceUpdater	Gas Optimization	Info	Pending
I-03	State Change Validation In The USH Oracle	Validation	Info	Pending
I-04	Overly Restrictive Approve Functionality	Logical Error	Info	Pending

Findings & Resolutions

ID	Title	Category	Severity	Status
I-05	Misleading Name Of maxDailyChangePercentage	Error	Info	Pending
I-06	Authentication Checked Twice For Msg.sender	Gas Optimization	Info	Pending
I-07	Redundant If-Else Branching In _update	Gas Optimization	Info	Pending
I-08	Batches Report Input Length	Informational	Info	Pending
I-09	Redundant Code Comment	Informational	Info	Acknowledged
I-10	updateChainalysisOracle Should Set Enabled	Informational	Info	Acknowledged
I-11	Warning About Burner Role	Warning	Info	Acknowledged
I-12	Consider Overriding renounceRole In USHToken	Unexpected Behavior	Info	Pending
I-13	Consider EIP Compliance	Best Practices	Info	Pending
I-14	AuthManager.sol Ignores maxBatchSize	Best Practices	Info	Pending
I-15	Unused Events/Errors	Gas Optimization	Info	Pending
I-16	Redundant Decimals Function In USHToken	Best Practices	Info	Pending
I-17	Misleading Comment	Informational	Info	Pending

Findings & Resolutions

ID	Title	Category	Severity	Status
I-18	Misleading PriceUpdated Event In Initialize	Best Practices	Info	Pending
I-19	Consider Using ERC20Votes For Governance	Informational	Info	Acknowledged

H-01 | AuthHook's Authentication Can Be Bypassed

Category	Severity	Location	Status
Trust Assumptions	High	AuthHook.sol: 149	Resolved

Description PoC

The `_checkSender` on `AuthHook.sol` is correctly trying to retrieve the user by `msgSender()`. However this practice must be done only if the periphery interacting with `PoolManager` is trusted.

Otherwise, the `msgSender()` can be manipulated to return any address.

Recommendation

Use a trusted pool of periphery contracts that will be calling `PoolManager` as stated in Uniswap v4 docs. Otherwise, do not allow the execution.

Resolution

Manifest Team: Resolved.

M-01 | Bypass KYC Check By Transferring

Category	Severity	Location	Status
Validation	Medium	StakedUSHBase.sol: 277-288	Acknowledged

Description PoC

There is no KYC check in the `_update` function of `StakedUSHBase.sol`. If a user's KYC is revoked, they cannot call `cooldownAssets` since it will revert.

However, they can still call `transfer` to move their tokens to another KYC'd account, which can then call `cooldownAssets`.

Recommendation

Remove in `_checkRestrictions` the `checkSanctioned` and `checkBanned`, and use `checkUser` instead. Otherwise, acknowledge this behavior as an acceptable risk and ensure it is documented.

Resolution

Manifest Team: Acknowledged.

M-02 | sUSH Cannot Be Redistributed

Category	Severity	Location	Status
Validation	Medium	StakedUSHBase.sol: 150-157	Resolved

Description PoC

sUSH holders may be banned or sanctioned. When either status applies, users are blocked from depositing/withdrawing/transferring sUSH.

Separately, the sUSH admin can redistribute a restricted user's balance (either burn to rewards or mint to another user) via `redistributeLockedAmount`, which is documented as:

```
The address to burn the entire balance which is restricted (banned or sanctioned)
```

However, `redistributeLockedAmount` only checks `isBanned` for `from` and `to`, and does not check the sanctioned status.

As a result, sanctioned users are treated as unrestricted in this path, causing the function to revert (since `isFromRestricted` is false) and effectively preventing admins from redistributing sanctioned users' sUSH, and leaving them stuck in the users' wallets.

Recommendation

Include `isSanctioned` check for both from and to users in `redistributeLockedAmount`.

Resolution

Manifest Team: Resolved.

M-03 | Price Decimal Mismatch In USHPriceOracle

Category	Severity	Location	Status
Unexpected Behavior	Medium	USHPriceOracle.sol: 39-43	Resolved

Description

The USHPriceOracle contract will be used to update USHToken prices, which are expected to have 8 decimals based on comments in the contract.

However, the MIN_PRICE (1e4) and MAX_PRICE (1e12) constants are defined based on 6-decimal price values.

If the minimum and maximum price constants are correct and the updater submits prices using 6-decimal precision, a 100x discrepancy will occur since external integrators expect 8-decimal precision.

Conversely, if the updater submits prices using 8-decimal precision, external integrators will receive the correct values, but the effective minimum and maximum bounds will be significantly lower than intended:

the minimum price will be $0.0001 instead of $0.01, and the maximum price will be $10,000 instead of $1,000,000.

Recommendation

Resolve the discrepancy according to the intended decimal precision. If 8 decimals are expected as indicated by the comments, update the MIN_PRICE and MAX_PRICE constants to 1e6 and 1e14 respectively.

Resolution

Manifest Team: Resolved.

M-04 | Unauthorized Pool Initialization And Donations

Category	Severity	Location	Status
Access Control	Medium	AuthHook.sol: 106-115	Resolved

Description

The `AuthHook` does not have `beforeInitialize` permission.

If the hook deployment and the initialization of the Manifest-controlled permissioned UniV4 pool are not performed in the same transaction, any user can initialize the pool and set an arbitrary initial `sqrtPriceX96`, since the parameters required to calculate the `poolKey` can be known beforehand.

Similarly, the hook has `beforeDonate` permission set to false. Since the protocol does not want non-KYC'd users to interact with the permissioned UniV4 pool, this action should also be restricted.

Recommendation

Set both the `beforeInitialize` and `beforeDonate` permissions to true. `beforeInitialize` should allow only protocol admins to initialize the pool.

`beforeDonate` should either allow only KYC'd users, similar to other hooks, or disallow donations entirely, depending on the protocol's preferences.

Resolution

Manifest Team: Resolved.

L-01 | Invalid Permits Due To Version Mismatch

Category	Severity	Location	Status
Validation	Low	StakedUSHBase.sol: 82-83	Resolved

Description

The version in the `USHToken` contract is "1k", while the domain separator is calculated using the hardcoded version value "1".

As a result, permit signatures that use the correct version cannot be validated because the signature digest is different.

Similarly, the `StakedUSHTokenBase` contract inherits `ERC20Permit`. However, token symbol is passed in the constructor instead of token name.

Note that permit signatures can still be valid if users sign based on the public `DOMAIN_SEPARATOR`, even though the versions and names do not match. However, it will be invalid if users sign it based on the correct version.

Recommendation

Ensure that the constant version value matches the version used in the domain separator.

Resolution

Manifest Team: Resolved.

L-02 | Inconsistent Behavior At Unstake

Category	Severity	Location	Status
Best Practices	Low	StakedUSH.sol: 104	Resolved

Description

If `cooldownDuration` is set to zero, all users can unstake immediately. However, if it is only reduced (not zero), users who started cooldown earlier remain subject to the longer duration, while new users benefit from the shorter one.

This creates an unfair disadvantage for unaware users who don't recall `cooldownAssets` after the parameter change.

Recommendation

Store both the `block.timestamp` and the `cooldownDuration` active at that time. When checking for unstake eligibility, allow users to unstake at the earlier of the two values (`stored block.timestamp + stored cooldownDuration` or `stored block.timestamp + new cooldownDuration`).

This ensures fairness and consistency when cooldown parameters change.

Resolution

Manifest Team: Resolved.

L-03 | No Cancellation Mechanism

Category	Severity	Location	Status
Informational	Low	StakedUSH.sol: 117	Acknowledged

Description

When a user initiates a cooldown via `cooldownAssets` or `cooldownShares`, The contract records a future `cooldownEnd` timestamp and increases the `underlyingAmount` held in the cooldown mapping.

However, the current design does not allow users to undo or adjust these requests. Once a cooldown is started:

The expiry time can only be extended further by making additional cooldown calls. If a user changes their mind (they no longer want to unstake), there is no way to cancel or scale down the request. The system forces them to complete the full cooldown process.

Recommendation

Introduce functionality that lets users reverse or reduce cooldown commitments.

Resolution

Manifest Team: Acknowledged.

L-04 | Unstake Cooldown Is Temporarily Denied

Category	Severity	Location	Status
DoS	Low	StakedUSH.sol: 130	Acknowledged

Description

Both `cooldownAssets` and `cooldownShares` send the assets to the silo as the `receiver`:

```
// StakedUSH.sol
_withdraw(msg.sender, address(silo), msg.sender, assets, shares);
```

In the base implementation, the auth layer enforces KYC on the receiver inside the shared withdraw path via `onlyAuthApproved(receiver) > authManager.checkUser(receiver)`.

Since the Silo is a system contract that isn't KYC'd, the call reverts until it has been granted KYC through the `grantKYC` function, causing a temporary DoS when starting the cooldown.

Recommendation

Ensure that the Silo contract is KYC'd by invoking the `grantKYC` function, either within the deployment script or manually immediately after deployment.

Resolution

Manifest Team: Acknowledged.

L-05 | Discrepancy Between Similar Functions

Category	Severity	Location	Status
Unexpected Behavior	Low	AuthManager.sol: 381-383	Resolved

Description

The `getKycStatus` function checks only the KYC status of an account and does not consider whether the account is banned.

In contrast, the `batchGetKycStatus` function first checks the banned status of the account and returns false regardless of the KYC status if the account is banned.

As a result, the KYC status of the same account may appear differently depending on which function is used.

Recommendation

Consider updating one of the functions based on the intended behavior so that both functions behave consistently.

Resolution

Manifest Team: Resolved.

L-06 | Unauthorized Users Can Perform Transfer

Category	Severity	Location	Status
Access Control	Low	StakedUSHBase.sol: 283-285	Resolved

Description

The `transferFrom` function in the `USHToken` contract checks the `from` and `to` addresses to ensure they are authorized. However, it does not check `msg.sender`.

As a result, a banned or sanctioned user can still interact with the protocol and transfer other users' funds if they have an allowance.

The same issue occurs in `_update` function of `StakedUSH` token as well.

Recommendation

Check `msg.sender` as well and block their interaction with the protocol.

Resolution

Manifest Team: Resolved.

L-07 | initializeVault Can Be Frontrunned

Category	Severity	Location	Status
Frontrunning	Low	StakedUSHBase.sol: 98-109	Resolved

Description

The StakedUSHBase contract has an initializeVault function that initially deposits 10,000 USHTokens to prevent donation attacks. This function can only be called by the owner when the totalSupply is 0.

However, there is no guarantee that this function will be the first deposit unless the deployment and initializeVault call are performed atomically.

A malicious user could front-run initializeVault and deposit only the MIN_SHARES amount of tokens. As a result, initializeVault would be blocked afterward because the totalSupply would no longer be zero.

Although this scenario could occur, it does not result in the same consequences as a traditional donation attack, as the contract enforces _checkMinShares and prevents regular users from minting zero shares.

Recommendation

Ensure that the deployment and initializeVault call are performed atomically. Note that this requires the deployer address to be the _owner specified in the constructor.

Alternatively, a check can be added to the _deposit function to revert if totalSupply is zero. This guarantees that initializeVault is executed as the first deposit, since regular deposits cannot occur until the owner mints the initial shares.

Resolution

Manifest Team: Resolved.

L-08 | requiresMultiSigApproval Misses Interval Check

Category	Severity	Location	Status
Validation	Low	USHPriceOracle.sol: 248-251	Resolved

Description

`requiresMultiSigApproval()` only checks whether the change exceeds the max percentage:

```
return _exceedsMaxChange(currentPrice, newPrice);
```

However, when the updater calls `updatePrice`, it can also revert on the time-based interval:

```
if (block.timestamp < s.lastUpdateTimestamp + s.minUpdateInterval) revert UpdateTooFrequent();
```

Therefore, `requiresMultiSigApproval` may return false (suggesting updater is fine) while an updater transaction would still revert due to the minimum interval check.

Recommendation

Consider including the interval check in `requiresMultiSigApproval`.

Resolution

Manifest Team: Resolved.

L-09 | Lost ECDSA Checks Enable Signature Malleability

Category	Severity	Location	Status
Validation	Low	SolmateERC20Upgradeable.sol: 201	Resolved

Description

The permit function in `SolmateERC20Upgradeable` uses `ecrecover` to validate signatures without implementing checks for signature malleability. It has no validation that s is in the lower half of the curve's order.

While the nonce mechanism (`nonces[owner]++`) prevents signature replay attacks, the lack of proper ECDSA checks remains a deviation from best practices.

Recommendation

It's recommended to require the `s` value to be in the lower half order.

Resolution

Manifest Team: Resolved.

L-10 | Reward Dilution Possible During transferInReward

Category	Severity	Location	Status
MEV	Low	StakedUSHBase.sol: 124-130	Acknowledged

Description

The StakedUSHBase.sol contract uses transferInRewards to distribute rewards linearly over an 8-hour vesting period.

This allows stakers to deposit before transferInRewards is called and withdraw after the vesting ends, effectively diluting rewards intended for long-term stakers.

For this dilution to be effective:

• cooldownDuration must be set to 0.

• cooldownDuration must not be increased during the 8-hour vesting window.

Recommendation

Distribute rewards in smaller, more frequent intervals, making dilution gains negligible. Allow the vesting period to be configurable when calling transferInRewards, enabling adjustments based on reward size.

Resolution

Manifest Team: Acknowledged.

L-11 | Missing Validation In Auth Batch Functions

Category	Severity	Location	Status
Validation	Low	AuthManager.sol: 292	Acknowledged

Description

The `AuthManager` contract correctly implements checks to prevent redundant state changes in its single-user functions like `banUser`, `removeBan`, `grantKyc`, and `revokeKyc`.

But, the corresponding batch functions (`batchBan`, `batchRemoveBan`, `batchGrantKyc`, `batchRevokeKyc`) lack these checks.

They unconditionally write to storage for every user in the provided array, even if a user's status is already the intended state. This inconsistency leads to unnecessary `SSTORE` operations in a loop.

Recommendation

Consider implementing the same check to skip the redundant state changes.

Resolution

Manifest Team: Acknowledged.

L-12 | Circulating Supply Misreported

Category	Severity	Location	Status
Unexpected Behavior	Low	USHToken.sol: 209-211	Resolved

Description

`USHToken.sol` inherits from `SolmateERC20Upgradeable`, whose `ERC20` implementation allows direct transfers to `address(0)` without decreasing `totalSupply`.

Meanwhile, `USHToken.sol` exposes a `circulatingSupply` function that returns `totalSupply`. This means that tokens transferred to `address(0)` are still counted, leading to an incorrect circulating supply calculation.

Recommendation

Prevent transfers to `address(0)` or update `circulatingSupply` to exclude balances at `address(0)` from its calculation.

Resolution

Manifest Team: Resolved.

L-13 | Checks Can Be Bypassed

Category	Severity	Location	Status
Warning	Low	USHToken.sol: 200	Acknowledged

Description

The `setAuthManager` function in the `USHToken` contract updates the `authManager` address, which is used to determine authorized users.

Although it is an owner-only function, sanctioned and banned checks can be bypassed during an update if the new `authManager` contract does not yet reflect the sanction and ban status maintained by the previous manager.

If an update is to occur, the new contract should persist the state of the previous contract before being set via the `setAuthManager` call.

Recommendation

Ensure that the new `authManager` contract preserves the state of the previous one before the update.

Resolution

Manifest Team: Acknowledged.

L-14 | setAuthManager Restarts Timelock

Category	Severity	Location	Status
Unexpected Behavior	Low	AuthHook.sol: 75	Acknowledged

Description

The `setAuthManager` function must be called twice to set the manager. The first call designates the pending manager and starts the timelock, while the second call, made after the timelock period, finalizes the manager.

However, if the second call is made before the timelock ends, it resets the pending manager and restarts the timelock, regardless of whether the provided address is the same as the current pending manager or a different one.

Recommendation

While this is an `onlyOwner` function, consider adding a check in the `else` block so that `_setAuthManagerReqTimestamp` is updated only if the provided address is different from `_pendingAuthManager`.

Resolution

Manifest Team: Acknowledged.

L-15 | hasValidAuth Does Not Account For Sanctions

Category	Severity	Location	Status
Validation	Low	AuthHook.sol: 96-102	Resolved

Description

In `AuthHook.sol`, the internal `_checkSender` function validates users by checking whether they are banned, sanctioned, or have KYC. This validation is enforced at `_beforeSwap`, `_beforeRemoveLiquidity`, and `_beforeAddLiquidity`.

However, the external view function `hasValidAuth` only checks for `hasKyc` and `isBanned`, not the sanctioned status. As a result, `hasValidAuth` can return `true` for a sanctioned user, even though `_checkSender` would revert when the same user interacts with the pool.

This creates an inconsistency between the view function and the actual logic.

Recommendation

Update the `hasValidAuth` view function to also verify the sanctioned status of the user, ensuring consistency with the `_checkSender` logic.

Resolution

Manifest Team: Resolved.

L-16 | MIN_PRICE And MAX_PRICE Limit Price Updates

Category	Severity	Location	Status
Best Practices	Low	USHPriceOracle.sol: 145	Resolved

Description

In `USHPriceOracle.sol`, two constants `MIN_PRICE` and `MAX_PRICE` define absolute lower and upper bounds for the price.

The contract also enforces additional constraints, such as minimum/maximum update intervals and minimum/maximum percentage change per update.

Unlike these other constraints, which can be bypassed by the `multiSigAddress`, the `MIN_PRICE` and `MAX_PRICE` bounds cannot be overridden.

This design may lead to unwanted situations where the protocol cannot update the oracle price to reflect real market conditions, even if the value falls outside of the hard-coded thresholds.

If the primary security concern is preventing a compromised `priceUpdater` from setting arbitrary values, the interval and percentage checks already provide sufficient safeguards.

In contrast, the hard-coded min/max values could unnecessarily block the protocol from setting a valid price in the future.

Recommendation

Consider making the `MIN_PRICE` and `MAX_PRICE` thresholds adjustable, or allow them to be bypassed by the `multiSigAddress`. This ensures that the protocol can adapt to changing market conditions.

Resolution

Manifest Team: Resolved.

L-17 | New Cooldown Resets cooldownEnd

Category	Severity	Location	Status
Best Practices	Low	StakedUSH.sol: 146	Acknowledged

Description

When cooldown is enabled, users must call `cooldownAssets` / `cooldownShares` to begin the unlock timer, and later call `unstake` to withdraw once the timer expires.

Both cooldown entry points overwrite the user's single cooldown timer:

```
// in cooldownAssets / cooldownShares
cooldowns[msg.sender].cooldownEnd = uint104(block.timestamp) + cooldownDuration;
cooldowns[msg.sender].underlyingAmount + uint152(assets);
```

This resets `cooldownEnd` to "now + duration" on every new request and aggregates the amounts. If a user starts a cooldown (e.g., for 100 tokens) and later starts another cooldown (another 100), the second call extends the wait for the entire balance (200) to the new, later `cooldownEnd`.

Recommendation

Consider refactoring the cooldown behavior to allow multiple cooldowns, which requires decent refactoring or document the current behavior clearly for users.

Resolution

Manifest Team: Acknowledged.

I-01 | priceUpdater Can Set expectedIndex = Uint256.max

Category	Severity	Location	Status
Best Practices	Info	USHPriceOracle.sol: 143	Resolved

Description

There is no limit on what value `expectedIndex` can take. It only checks that it is greater than the old index.

However, the `priceUpdater` could set it to `type(uint256).max`, making it impossible to call the `updatePrice` function again.

Recommendation

Ensure that `expectedIndex` is greater than the old index, but only by 1.

Resolution

Manifest Team: Resolved.

I-02 | Move _exceedsMaxChange Inside priceUpdater

Category	Severity	Location	Status
Gas Optimization	Info	USHPriceOracle.sol: 151	Resolved

Description

The `_exceedsMaxChange` function checks whether `newPrice` has moved beyond `maxDailyChangePercentage` compared to `currentPrice`.

It returns `true` if the limit is exceeded, otherwise `false`. However, the return value is only used when the caller is `priceUpdater`, making it useless in the case of `multiSigAddress`.

Recommendation

Move `_exceedsMaxChange` inside the `if` block that checks whether the caller is `priceUpdater`

Resolution

Manifest Team: Resolved.

I-03 | State Change Validation In The USH Oracle

Category	Severity	Location	Status
Validation	Info	USHPriceOracle.sol: 199	Resolved

Description

The contract includes checks to prevent redundant state changes in some setter functions like `setPriceUpdater`, `setMultiSigAddress`, and `setEmergencyPause` by reverting with "`AlreadyInThisState`".

This is a good practice as it avoids unnecessary storage writes and event emissions.

However, `setMaxDailyChangePercentage` and `setMinUpdateInterval` functions do not have the same checks.

Recommendation

Consider adding the check to `setMaxDailyChangePercentage` and `setMinUpdateInterval` to ensure the new value is different from the existing one.

Resolution

Manifest Team: Resolved.

I-04 | Overly Restrictive Approve Functionality

Category	Severity	Location	Status
Logical Error	Info	USHToken.sol: 154-156	Resolved

Description

The approve function in the `USHToken` contract include the `whenNotPaused` modifier. While it makes sense to use `whenNotPaused` for actual transfer functions, applying it to approve overly restricts users.

This is particularly important when users want to revoke their approvals during a paused period. In the event of a contract pause, users may reasonably want to revoke their approvals as a precautionary measure. Restricting this action during a paused state results in poor user experience.

The `OpenZeppelin` extension of `ERC20Pausable` only pauses the `_update` function, which affects transfers and minting/burning, but does not affect approvals (Reference).

Note that the same issue exists in the permit function as well.

Recommendation

Consider removing the `whenNotPaused` modifier from `approve` to allow users to revoke their approvals even during a paused state. Alternatively, ensure this behavior is explicitly documented to inform users.

Resolution

Manifest Team: Resolved.

I-05 | Misleading Name Of maxDailyChangePercentage

Category	Severity	Location	Status
Error	Info	USHPriceOracle.sol: 100	Resolved

Description

The variable `maxDailyChangePercentage` in `USHPriceOracle` enforces the maximum price change per update, not per day.

The current name implies that the restriction applies over a 24-hour period, which is misleading.

Recommendation

Rename the variable to reflect its true behavior.

Resolution

Manifest Team: Resolved.

I-06 | Authentication Checked Twice For Msg.sender

Category	Severity	Location	Status
Gas Optimization	Info	StakedUSH.sol: 120	Resolved

Description

In `StakedUSH.sol`, the functions `cooldownAssets` and `cooldownShares` use the `onlyAuthApproved(msg.sender)` modifier to check if the caller is approved. However, both functions later call `_withdraw`, which again validates `caller`, `owner`, and `receiver`.

This results in redundant authentication checks, since the same validation is effectively performed twice.

Recommendation

Remove the `onlyAuthApproved(msg.sender)` modifier from `cooldownAssets` and `cooldownShares` to avoid redundant checks.

Resolution

Manifest Team: Resolved.

I-07 | Redundant If-Else Branching In _update

Category	Severity	Location	Status
Gas Optimization	Info	StakedUSHBase.sol: 277-288	Resolved

Description

In StakedUSHBase.sol, the _update function uses an if-else statement to handle zero-address transfers.

This structure is redundant because super._update is always executed, and the only difference is whether restrictions are checked. The branching reduces readability without changing functionality.

Recommendation

Refactor the function to use a single if statement, checking restrictions only when both from and to are not the zero address. This simplifies the flow and avoids unnecessary branching.

Resolution

Manifest Team: Resolved.

I-08 | Batches Report Input Length

Category	Severity	Location	Status
Informational	Info	AuthManager.sol: 124	Acknowledged

Description

In the wallets batch updaters for example, the code emit `BatchAuthorizedWalletsAdded(length)` / `BatchAuthorizedWalletsRemoved(length)` using the input array length, even if some entries are duplicates or already in the desired state.

That means the event can claim “N added/removed” when fewer wallets actually changed.

Recommendation

Only count and emit per-wallet events when a flip actually occurs. The same issue exists in the batch-ban and batch-kyc functions.

Resolution

Manifest Team: Acknowledged.

I-09 | Redundant Code Comment

Category	Severity	Location	Status
Informational	Info	SanctionsList.sol: 20-23	Acknowledged

Description

```
/**
* @title IChainalysisOracle
* @dev Interface for the official Chainalysis Oracle at
0x40c57923924b5c5c5455c48d93317139addac8fb
*/
```

The comment above appears in lines 20–23 of the `SanctionsList` contract, but it should belong to the `IChainalysisOracle` interface.

Recommendation

Remove the comment from the `SanctionsList` contract and add it to the `IChainalysisOracle` interface.

Resolution

Manifest Team: Acknowledged.

I-10 | updateChainalysisOracle Should Set Enabled

Category	Severity	Location	Status
Informational	Info	SanctionsList.sol: 161-163	Acknowledged

Description

The `updateChainalysisOracle` function updates the oracle address but does not set `chainalysisEnabled` to true if it was previously false.

When `chainalysisEnabled` is false, setting a new oracle requires two calls: `updateChainalysisOracle` and `setChainalysisEnabled`.

Recommendation

Consider setting `chainalysisEnabled` to true within `updateChainalysisOracle` when updating the oracle.

Resolution

Manifest Team: Acknowledged.

I-11 | Warning About Burner Role

Category	Severity	Location	Status
Warning	Info	USHToken.sol: 119-122	Acknowledged

Description

The `BURNER_ROLE` in the `USHToken` contract can burn any amount from any user without restrictions. This behavior should be clearly documented for users.

Note that the sole purpose of this finding is to inform users.

Recommendation

No fix is required, as this is a design choice of the protocol. It is recommended that this behavior be documented, and the finding can be acknowledged.

Resolution

Manifest Team: Acknowledged.

I-12 | Consider Overriding renounceRole In USHToken

Category	Severity	Location	Status
Unexpected Behavior	Info	USHToken.sol	Resolved

Description

The StakedUSH contract overrides the renounceRole function and does not allow this operation.

However, this is not the case in the USHToken contract, and any role including the admin role can be renounced.

Additionally, the contract does not check for a zero address when setting the defaultAdmin in the initialize function.

Recommendation

Consider overriding renounceRole for at least the defaultAdmin role, and adding a zero address check in the initialize function

Resolution

Manifest Team: Resolved.

I-13 | Consider EIP Compliance

Category	Severity	Location	Status
Best Practices	Info	Global	Resolved

Description

The protocol implements a namespaced storage layout for its upgradeable contracts. Each contract has a storage location derived from the keccak hash of its name, such as `keccak256("storaged.ush.token")` or `keccak256("v1.storage.sanctions.list.manifest.finance")`.

While this approach prevents storage collisions between contracts, it is considered best practice to use the storage location formula from ERC-7201 for namespaced storage layouts, which involves double hashing and masking with ~0xff.

Recommendation

Consider using the `ERC-7201` formula to determine storage locations as a best practice.

Resolution

Manifest Team: Resolved.

I-14 | AuthManager.sol Ignores maxBatchSize

Category	Severity	Location	Status
Best Practices	Info	AuthManager.sol: 113	Resolved

Description

In `AuthManager.sol`, there is a storage variable `maxBatchSize` that defines the maximum number of updates allowed in a single batch.

This variable is supposed to control batch limits across all `batch` functions. However, the implementation directly uses the constant `MAX_BATCH_SIZE` instead of using the storage variable.

Recommendation

Make use of the `maxBatchSize` storage variable in all relevant `batch` functions instead of the `MAX_BATCH_SIZE` constant, ensuring smooth upgradability.

Resolution

Manifest Team: Resolved.

I-15 | Unused Events/Errors

Category	Severity	Location	Status
Gas Optimization	Info	IUSHToken.sol	Resolved

Description

Unused errors in `IUSHToken.sol` interface:

- `TooFrequentBurn`
- `TooExcessiveBurn`
- `InsufficientBalance`
- Sanctioned

Unused events in `IUSHToken.sol` interface:

- Sanction
- `MintFailure`
- `BurnFailure`

Unused errors in `AuthHook.sol` contract

- `OnlyAuthedUser`
- `UserBanned`
- `InvalidAuthManager`
- `TimelockNotExpired`
- `PendingManagerMismatch`

Recommendation

Remove unused events/errors

Resolution

Manifest Team: Resolved.

I-16 | Redundant Decimals Function In USHToken

Category	Severity	Location	Status
Best Practices	Info	USHToken.sol: 94-98	Resolved

Description

The `decimals` function in `USHToken` is redundant because the number of decimals is already set in storage by inheriting from `SolmateERC20Upgradeable`, which already implements the decimals function that returns the stored value.

Recommendation

Remove the `decimals` function from `USHToken`.

Resolution

Manifest Team: Resolved.

I-17 | Misleading Comment

Category	Severity	Location	Status
Informational	Info	USHToken.sol: 207	Resolved

Description

The comment on the `circulationSupply` function in the `USHToken` contract states, "Returns total supply less reserves," but this is misleading because the function always returns the total supply.

Recommendation

Update the comment.

Resolution

Manifest Team: Resolved.

I-18 | Misleading PriceUpdated Event In Initialize

Category	Severity	Location	Status
Best Practices	Info	USHPriceOracle.sol: 105	Acknowledged

Description

In `USHPriceOracle.sol`, the `initialize` function emits the `event PriceUpdated(uint256 indexed newPrice, uint256 oldPrice, address indexed updater)` event.

However, the `updater` field is set to `address(this)` instead of the actual caller (`msg.sender`) or the designated `owner/updater` address.

Emitting `address(this)` as the updater does not provide meaningful information, since the contract itself cannot directly act as the updater.

Recommendation

Update the event emission to use either `msg.sender` or the explicitly provided `updater`/`owner` address to ensure the event accurately reflects the responsible for initializing the price.

Resolution

Manifest Team: Acknowledged.

I-19 | Consider Using ERC20Votes For Governance

Category	Severity	Location	Status
Informational	Info	Global	Acknowledged

Description

According to the documentation of the protocol, `USAToken` will serve as the governance token.

Additionally, it states that `USHToken` holders can vote to initiate the redemption process for real-world assets.

However, both of these tokens are regular `ERC20` tokens without the `ERC20Votes` implementation.

While it is possible to use standard `ERC20` tokens for governance voting through off-chain mechanisms, the `ERC20Votes` implementation provides additional features such as delegation, checkpoints, and historical voting power data.

It is important to note that incorporating this feature would add complexity to the protocol, and the decision should ultimately depend on protocol-specific requirements.

Recommendation

Consider using `ERC20Votes` if on-chain voting data and delegation are important for the protocol.

Otherwise, ensure that the off-chain voting mechanism includes snapshots or other safeguards to prevent manipulations such as buy-vote-sell immediately.

Resolution

Manifest Team: Acknowledged.

Remediation Findings & Resolutions

ID	Title	Category	Severity	Status
M-01	KYC Bypass Via EIP7702	Validation	Medium	Acknowledged
M-02	Even manifestAccounts Cannot Donate	Validation	Medium	Resolved
L-01	Hardcoded Addresses Restrict Future Expansion	Suggestion	Low	Resolved
L-02	Misleading Comment Regarding Price Oracle	Compatibility	Low	Resolved
L-03	Cast To Uint256 When Comparing Lower Half	Validation	Low	Resolved
L-04	Non-Standard Events Break ERC20 Compatibility	Compatibility	Low	Resolved
L-05	ERC20Permit Domain Name Mismatch	Compatibility	Low	Resolved
L-06	Misleading ERC4626 View Methods On Deposit	Compatibility	Low	Resolved
L-07	Missing View In IAuthManager	Suggestion	Low	Resolved
L-08	Positions Cannot Be Minted Via UniversalRouter	Warning	Low	Acknowledged
L-09	Warning About KYC Restrictions	Warning	Low	Acknowledged
I-01	Division By Zero In _exceedsMaxChange	Warning	Info	Resolved
I-02	Owner Can Set Inconsistent Bounds	Validation	Info	Resolved

Remediation Findings & Resolutions

ID	Title	Category	Severity	Status
I-03	Redundant Check In updatePrice	Superfluous Code	Info	Resolved
I-04	Misleading NatSpec Comment On checkBanned	Informational	Info	Resolved
I-05	Incorrect Comment On maxChangePercentage	Informational	Info	Resolved
I-06	Inconsistent Checks In Initialization Callback	Validation	Info	Resolved
I-07	Pending Auth Manager Lacks Reset Option	Best Practices	Info	Resolved
I-08	Important Bound Changes	Best Practices	Info	Resolved
I-09	UniV4 Quotes Always Fail	Unexpected Behavior	Info	Resolved

M-01 | KYC Bypass Via EIP7702

Category	Severity	Location	Status
Validation	Medium	Global	Acknowledged

Description

With EIP-7702, any KYC'd user can set their account code to allow non-KYC users to interact with the protocol, effectively bypassing the KYC check.

Normally, non–KYC'd users can hold and transfer USHToken, but they cannot interact with the permissioned Uniswap pool or perform swaps.

However, this restriction can be bypassed with EIP-7702. Consider the following scenario:

• Alice is not KYC'd

• Bob is KYC'd

• Bob sets his account code using EIP-7702 to execute a swap on the Uniswap pool (Bob's account calls UniversalRouter).

• Alice calls Bob's account and executes the swap through it.

• Since msgSender in UniversalRouter is Bob's account, the KYC checks are bypassed.

In this way, a single KYC'd user can enable all other non-KYC users to interact with the protocol.

Recommendation

One option is to also check the KYC status of tx.origin. However, this could introduce new restrictions and may lead to unexpected behaviors.

Another option is to monitor users off-chain and ban those who behave in this way.

Resolution

Manifest Team: Acknowledged. We will track this off-chain.

M-02 | Even manifestAccounts Cannot Donate

Category	Severity	Location	Status
Validation	Medium	AuthHook.sol: 173	Resolved

Description

The `_beforeDonate` hook is designed to allow `manifestAccounts` to donate while preventing any other accounts from doing so.

The hook first checks whether the sender is one of the allowed accounts via `_checkAllowed`, and then tries to get the `msgSender` from it.

However, none of the hardcoded addresses, including the `UniversalRouter` and `PositionManager`, support the donate functionality. Reference.

The only way to donate is by directly calling the `PoolManager` or using custom routers. As a result, `_checkAllowed` always reverts during `_beforeDonate`.

Recommendation

If donations are expected to be allowed for `manifestAccounts`, either a custom router must be implemented and verified in `_checkAllowed`, or the `manifestAccounts` should directly call the `PoolManager`.

In the latter case, the `sender` should not be checked via `_checkAllowed` but should instead be validated directly using `checkManifestAccount`.

Resolution

Manifest Team: The issue was resolved in commit 7cff6c8.

L-01 | Hardcoded Addresses Restrict Future Expansion

Category	Severity	Location	Status
Suggestion	Low	AuthHook.sol	Resolved

Description

The `AuthHook` enforces a fixed set of hardcoded addresses as trusted senders. However, this restricts the protocol if expansion to other chains is required or if Uniswap introduces new routers.

Recommendation

Consider storing the trusted senders in a mapping, and implementing owner-only setter functions for flexibility.

Resolution

Manifest Team: The issue was resolved in commit 7cff6c8.

L-02 | Misleading Comment Regarding Price Oracle

Category	Severity	Location	Status
Compatibility	Low	USHPriceOracle.sol	Resolved

Description

The min and max prices in the `USHPriceOracle` are based on 6-decimal values, and the `priceUpdater` or multisig updates the prices using the same 6-decimal format.

However, the comments in the `USHPriceOracle` and `USHPriceOracleStorage` contracts, as well as in the `IUSHPriceOracle` interface, still indicate that prices use 8 decimals, which is misleading for external integrators.

Recommendation

Update all comments to ensure that integrators receive the correct price information.

If the prices are intended to use 8 decimals, like Chainlink, the previous issue remains, and the min/max prices need to be updated accordingly.

Resolution

Manifest Team: The issue was resolved in commit d1a4e3e.

L-03 | Cast To Uint256 When Comparing Lower Half

Category	Severity	Location	Status
Validation	Low	SolmateERC20Upgradeable.sol: 202-203	Resolved

Description

As a fix for the previous L-09 issue, the `s` value is checked to ensure it is in the lower half of the curve order. However, this check is performed directly on the `bytes32` value without converting it to `uint256`.

As a result, a lexicographic comparison is used instead of a numeric one, which could lead to incorrect behavior if the byte order is misinterpreted.

Recommendation

Cast the s value to `uint256` before comparing it, as done in the OpenZeppelin library

```
if (uint256(s) > 0x7FFFFFFFFFFFFFFFFFFFFFFFFFFFFFFF5D576E7357A4501DDFE92F46681B20A0) { revert
}
```

Resolution

Manifest Team: The issue was resolved in commit c70a1d0.

L-04 | Non-Standard Events Break ERC20 Compatibility

Category	Severity	Location	Status
Compatibility	Low	SolmateERC20Upgradeable.sol: 22-27	Resolved

Description

`Transfer` and `Approval` events in `SolmateERC20Upgradeable` declare the amount parameter as indexed. While syntactically valid, this deviates from the `ERC-20` standard which defines amount as non-indexed.

Many off-chain indexers, wallets, and analytics tools assume amount is encoded in the data section, not in a topic; this will break integrations and monitoring relying on the standard ABI encoding.

Recommendation

Change Transfer and Approval events to the `ERC-20` standard form (Reference).

Resolution

Manifest Team: Resolved.

L-05 | ERC20Permit Domain Name Mismatch

Category	Severity	Location	Status
Compatibility	Low	StakedUSHBase.sol: 82	Resolved

Description

The version mismatch in the `USHToken` contract from the previous L-01 issue has been fixed; however, the name mismatch in `StakedUSHTokenBase` still persists.

The `ERC20` name is set to "Staked USH" while `ERC20Permit` is initialized with the name "sUSH".

`EIP-2612` requires the `EIP-712` domain name to match the `ERC20` name; a mismatch causes off-chain signatures to be computed over a different domain than the contract enforces, making permit signatures fail or be non-standard.

Recommendation

Initialize `ERC20Permit` with the same name string used by `ERC20`. For example: `ERC20("Staked USH", "sUSH"); ERC20Permit("Staked USH");`

Resolution

Manifest Team: The issue was resolved in commit a14393e.

L-06 | Misleading ERC4626 View Methods On Deposit

Category	Severity	Location	Status
Compatibility	Low	StakedUSHBase.sol,637	Resolved

Description

Deposits are disallowed until the admin performs `initializeVault()` because `_deposit` requires `totalSupply() = 0`.

However, standard `ERC4626` views like `maxDeposit/previewDeposit` do not reflect this and may suggest deposits are possible, causing integrators to attempt deposits that always revert.

Recommendation

Override `maxDeposit` and/or `previewDeposit` to return 0 or otherwise signal unavailability when `totalSupply() = 0`.

Alternatively, this can be acknowledged if the protocol intends to initialize the vault atomically or immediately after deployment.

Resolution

Manifest Team: Resolved. We handled this in the deployment script.

L-07 | Missing View In IAuthManager

Category	Severity	Location	Status
Suggestion	Low	IAuthManager.sol: 130	Resolved

Description

`checkUser` function in `IAuthManager` interface is declared without the view modifier, despite documentation explicitly requiring it remain view-only.

Recommendation

Mark `IAuthManager.checkUser` as external view in the interface.

Resolution

Manifest Team: The issue was resolved in commit e914b89.

L-08 | Positions Cannot Be Minted Via UniversalRouter

Category	Severity	Location	Status
Warning	Low	AuthHook.sol	Acknowledged

Description

KYC'd users will interact with UniversalRouter to perform swaps, while manifestAccounts will interact with PositionManager to add or remove liquidity.

However, UniversalRouter allows minting positions via PositionManager but does not support liquidity adjustments or position burns. (Reference1, Reference2).

```
// should only call modifyLiquidities() to mint
_checkV4PositionManagerCall(inputs);
(success, output) = address(V4_POSITION_MANAGER).call{value: address(this).balance}(inputs);
```

During this call, UniversalRouter is the msg.sender of the PositionManager contract. Since PositionManager is one of the allowed senders in the AuthHook, _getSender(sender) returns the UniversalRouter address, and the action reverts because that address is not a manifestAccount.

However, adding this router to the manifestAccounts would bypass the entire check and allow anyone, including non-KYC'd users, to mint positions through the router.

The current behavior of disallowing position minting through UniversalRouter is more aligned with the intended restriction logic; however, manifestAccounts should be aware of this limitation.

Recommendation

Be aware of this behavior and ensure that manifestAccounts always interact with PositionManager directly, rather than through UniversalRouter, for any liquidity-related actions.

Additionally, ensure that UniversalRouter is never added to the list of manifestAccounts, as this would bypass the intended restrictions.

Resolution

Manifest Team: Acknowledged. UniversalRouter should never be granted ManifestAccount permissions.

L-09 | Warning About KYC Restrictions

Category	Severity	Location	Status
Warning	Low	AuthHook.sol	Acknowledged

Description

Only KYC'd users can swap tokens in the permissioned Uniswap pool. While the `SETTLE_ALL` and `TAKE_ALL` actions use the `msgSender` address, the `SETTLE` and `TAKE` actions accept arbitrary addresses.

These addresses can be provided to the `UniversalRouter` by the caller and then retrieved using the `_mapPayer` and `_mapRecipient` internal functions on the router side. Reference.

A KYC'd user can perform swaps on behalf of non-KYC'd users. While this is similar to a KYC'd user executing a swap themselves and then transferring the tokens to a non-KYC'd user, in this case, the funds are exchanged directly between Uniswap and the non-KYC'd user.

Recommendation

Be aware of this behavior. If this is not acceptable, unlike allowing a non-KYC'd user to hold and/or transfer tokens, this action may also need to be restricted.

However, the recipient address is not part of the `SwapParams`, so this cannot be restricted in the hook. It is a feature of the router. As a result, if this action must be restricted, it needs to be enforced at the token transfer level.

Resolution

Manifest Team: Acknowledged. We will track this off-chain.

I-01 | Division By Zero In _exceedsMaxChange

Category	Severity	Location	Status
Warning	Info	USHPriceOracle.sol: 357	Resolved

Description

`_exceedsMaxChange` may revert with a panic error if the `currentPrice` is 0. Previously, this was not possible because `MIN_PRICE` was a constant.

However, with the updates, `MIN_PRICE` is now configurable and can be set to 0. While the owner is trusted, a compromised owner account could set this value to 0, which would break future price updates.

Recommendation

Enforce that `newMinPrice` is greater than 0 in the `set_MIN_PRICE` function.

Resolution

Manifest Team: The issue was resolved in commit 865b7f9.

I-02 | Owner Can Set Inconsistent Bounds

Category	Severity	Location	Status
Validation	Info	USHPriceOracle.sol	Resolved

Description

The owner can set MIN and MAX constraints to inconsistent values (e.g., `MIN_PRICE > MAX_PRICE` or `MIN_UPDATE_INTERVAL > MAX_UPDATE_INTERVAL`).

This makes all subsequent updates fail bound checks, effectively freezing oracle updates until the owner fixes the configuration.

Recommendation

Add validation in setters to ensure invariants hold, e.g., `require(newMin < current MAX)` and `require(newMax > current MIN)`.

Resolution

Manifest Team: The issue was resolved in commit 7aa8f89.

I-03 | Redundant Check In updatePrice

Category	Severity	Location	Status
Superfluous Code	Info	USHPriceOracle.sol: 133	Resolved

Description

The updatePrice function checks whether the expected index is greater than the current update counter to prevent old updates. There are two checks for this:

```
if (expectedIndex < s.updateCounter + 1) revert IndexMustBeGreater();
if (expectedIndex = s.updateCounter + 1) revert IndexMustGrowAsSequence();
```

However, the first check is redundant, as the second check is more restrictive and requires the index to be exactly the next one.

Recommendation

if (expectedIndex < s.updateCounter + 1) revert IndexMustBeGreater() check can be removed.

Resolution

Manifest Team: The issue was resolved in commit 728234b.

I-04 | Misleading NatSpec Comment On checkBanned

Category	Severity	Location	Status
Informational	Info	AuthManager.sol: 136	Resolved

Description

The `NatSpec` comment for the `checkBanned` function, in both the `AuthManager` contract and the `IAuthManager` interface, reads:

“Reverts with `InvalidAddress` if the account is the zero address, or `UserNotPermitted` if the user is banned.”

However, the function returns when the address is 0 and does not revert.

Recommendation

Update the comment.

Resolution

Manifest Team: The issue was resolved in commit 8f62397.

I-05 | Incorrect Comment On maxChangePercentage

Category	Severity	Location	Status
Informational	Info	IUSHPriceOracle.sol: 80	Resolved

Description

The comment on line 22 of the `USHPriceOracleStorage` contract and line 80 of the `IUSHPriceOracle` interface still indicates that the change is daily:

“Maximum allowed daily price change percentage.”

However, with the updates, this no longer refers to a daily price change but to the price change between each update.

Recommendation

Update comments.

Resolution

Manifest Team: The issue was resolved in commit 17e8705.

I-06 | Inconsistent Checks In Initialization Callback

Category	Severity	Location	Status
Validation	Info	AuthHook.sol: 126	Resolved

Description

In `AuthHook.sol`, the `_beforeInitialize` callback enforces a different access control pattern than the other callbacks.

Specifically, it calls `authManager.checkManifestAccount(sender)` directly, without first validating the sender through `_checkAllowed` and then `checkManifestAccount(_getSender(sender))`.

The team's intention is to allow pool initialization directly via the Uniswap `PoolManager.initialize` function when using a manifest account.

However, if the `PositionManager` contract itself is ever added to the manifest accounts, this design unintentionally permits any user to initialize a pool with this hook, effectively bypassing intended restrictions.

Recommendation

For uniformity and to prevent unauthorized pool initialization, apply the same validation logic in `_beforeInitialize` as used in other callbacks.

Resolution

Manifest Team: The issue was resolved in commit 7cff6c8.

I-07 | Pending Auth Manager Lacks Reset Option

Category	Severity	Location	Status
Best Practices	Info	AuthHook.sol: 56-76	Resolved

Description

In `AuthHook.sol`, the `setAuthManager` function allows the owner to set a new `pendingAuthManager`, update it if called with a different address, or finalize the update once the timelock period has elapsed.

However, there is currently no mechanism to cancel an existing pending authorization request.

Once a `pendingAuthManager` is set, the only way to clear it is by replacing it with another address and waiting through the timelock.

This could limit flexibility if the owner wants to abort an update without initiating a replacement.

Recommendation

Consider adding a method that allows the owner to cancel the current pending authorization request, resetting both `_pendingAuthManager` and `_setAuthManagerReqTimestamp` to their default values.

This would provide a clean way to abort an in-progress update safely.

Resolution

Manifest Team: The issue was resolved in commit a7c0a0a.

I-08 | Important Bound Changes

Category	Severity	Location	Status
Best Practices	Info	USHPriceOracle.sol	Resolved

Description

Owner-only setters that adjust the global bounds (MIN/MAX for price, interval, and change percentage) do not emit events.

Silent changes reduce transparency and can surprise off-chain systems. These setters should emit events as best practice.

Recommendation

Emit dedicated events for each adjustable parameter change, including old and new values.

Resolution

Manifest Team: The issue was resolved in commit e29f256.

I-09 | UniV4 Quotes Always Fail

Category	Severity	Location	Status
Unexpected Behavior	Info	AuthHook.sol,306	Resolved

Description

The Quoter address is hardcoded as an allowed sender, and authorization is checked via `_getSender(sender)`. However, the Uniswap Quoter does not implement the `msgSender()` function.

As a result, the try-catch will fail and default to treating the sender (i.e., the Quoter contract) as the user.

This leads to two scenarios. It is impossible to quote swaps without granting KYC to the Quoter contract.

However, if the Quoter is KYC'd, then everyone can interact with the protocol through the Quoter.

Recommendation

All official Quoter addresses on supported chains should be KYC'd in order to perform quotes. Also, be aware that there is no way to prevent non-KYC'd users from performing quotes without implementing your own Quoters.

Resolution

Manifest Team: Resolved. We handled this in the deployment script.

Disclaimer

This report is not, nor should be considered, an “endorsement” or “disapproval” of any particular project or team. This report is not, nor should be considered, an indication of the economics or value of any “product” or “asset” created by any team or project that contracts Guardian to perform a security assessment. This report does not provide any warranty or guarantee regarding the absolute bug-free nature of the technology analyzed, nor do they provide any indication of the technologies proprietors, business, business model or legal compliance.

This report should not be used in any way to make decisions around investment or involvement with any particular project. This report in no way provides investment advice, nor should be leveraged as investment advice of any sort. This report represents an extensive assessing process intending to help our customers increase the quality of their code while reducing the high level of risk presented by cryptographic tokens and blockchain technology.

Blockchain technology and cryptographic assets present a high level of ongoing risk. Guardian’s position is that each company and individual are responsible for their own due diligence and continuous security. Guardian’s goal is to help reduce the attack vectors and the high level of variance associated with utilizing new and consistently changing technologies, and in no way claims any guarantee of security or functionality of the technology we agree to analyze.

The assessment services provided by Guardian is subject to dependencies and under continuing development. You agree that your access and/or use, including but not limited to any services, reports, and materials, will be at your sole risk on an as-is, where-is, and as-available basis. Cryptographic tokens are emergent technologies and carry with them high levels of technical risk and uncertainty. The assessment reports could include false positives, false negatives, and other unpredictable results. The services may access, and depend upon, multiple layers of third-parties.

Notice that smart contracts deployed on the blockchain are not resistant from internal/external exploit. Notice that active smart contract owner privileges constitute an elevated impact to any smart contract’s safety and security. Therefore, Guardian does not guarantee the explicit security of the audited smart contract, regardless of the verdict.

About Guardian

Founded in 2022 by DeFi experts, Guardian is a leading audit firm in the DeFi smart contract space. With every audit report, Guardian upholds best-in-class security while achieving our mission to relentlessly secure DeFi.

To learn more, visit https://guardianaudits.com

To view our audit portfolio, visit https://github.com/guardianaudits

To book an audit, message https://t.me/guardianaudits